

OfficeQA Arena – Cohort 0 Final Report

1. Team Info

- **Team name:** Zero Node
- **Team members:** Tarun Reddi, Jwalin Shah, Sanjay Sai

2. What We Built

We built a grounded numerical QA system for answering 246 financial questions over 696 U.S. Treasury Bulletin text files. Our final submission uses a **minimal prompt (~20 lines) with no database, no MCP server, and no skills** – just MiniMax M2.5 with shell access to the raw corpus via the Goose harness. Tarball: 28KB. **Best score: 184.5/246 (75.0%) at \$1.71 total cost.**

Core idea: Let MiniMax grep raw text files directly instead of routing through structured tools. We tried 9 architectural generations across 43 arena runs – from an 11GB SQLite database with 7 MCP tools down to a bare 3-line prompt. *Simpler systems consistently outperformed complex ones.*

The data engineering journey: We built 5 database versions through a multi-stage ingestion pipeline (parsing, enrichment, compression, synthesis). The final DB had 677K master ledger records, 131K precomputed YoY changes, and 204K metric aliases. We also built an automatic solver that attempted to answer questions deterministically by decomposing them into structured schemas and routing through the database. This solver achieved 46% accuracy – not submittable, but invaluable as a diagnostic. It revealed exactly where data was breaking: multi-row headers splitting tables, footnote-contaminated values, 41% of tables with NULL year metadata, and the fact that calendar-year totals must be synthesized from 12 monthly values. The solver proved this is fundamentally a **data ingestion problem** – clean the corpus correctly and QA becomes straightforward arithmetic.

We also built a 10-component orchestrator pipeline (FileLocator, TableFinder, ColumnFinder, RowFinder, ValueParser, DataNormalizer, TableValidator, ConsensusVoter, SearchTask, Orchestrator) with three-strategy consensus voting. Component tests passed 103/103, but end-to-end results were nondeterministic and we never got to fully test it in the arena. The engineering was essential to understanding what can be automated away from the LLM.

What made it work:

- **Raw text over databases.** Our DB lost ~50% of cell data through parsing edge cases. Grep had 100% coverage.
- **Mentor/review prompt framing.** “Review your intern’s analysis” activated genuine critical evaluation (+13 points).
- **Inline reference data.** Embedding CPI-U values in the prompt eliminated hallucination (the model fabricated CPI numbers when API calls failed).

3. How We Worked On It

Iteration cycle: Submit -> pull traces within 24h (they get purged) -> classify failures -> build fix -> A/B test locally -> submit again. We built a local test harness replicating the arena environment with oracle page files.

- **Trace analysis was everything.** We analyzed ~4,400 task traces across 6 versions. MCP tools were *never called* in arena. Tool call count inversely correlates with success.
- **14-variant A/B test** on 40 tasks. 70% is MiniMax’s hard ceiling regardless of prompt. Prompt engineering’s entire value (~9%) is ensuring the model writes `answer.txt`.

- **Cross-model comparison.** 8 frontier models (Claude Sonnet/Haiku, GPT-5-mini/GPT-4o, MiniMax, Kimi K2, Gemini Flash, DeepSeek, Grok) on 20 tasks. 3/8 were completely incompatible with Goose's tool-calling format. Oracle ensemble of 3 models hit 75% vs 50% for any single model.

4. What We Found

Finding the right table is the real bottleneck. 48% of failures trace to the wrong table/row/column – not reasoning or arithmetic. Zero failures among correctly-grounded answers were arithmetic errors. This is compounded by the oracle vs. full corpus gap: oracle page files achieve 76.3% pass rate, but our search recall on the full 150MB corpus was only 47-63%. A real system without oracle hints faces a much harder retrieval problem.

Structured tools hurt. When MCP tools were available, MiniMax wasted 10-21 steps before falling back to grep. Forcing MCP usage via terminal blocking produced 0% accuracy.

MiniMax is action-triggered, not protocol-following. Only three prompt elements work: (1) "write answer.txt," (2) file path hints, (3) "use python3 -c." Negative instructions ("NEVER curl") *increase* the forbidden behavior.

Score variance dominates prompt effects. Same-config runs vary by 8-17 points. Prompt changes produce ~5 point differences – within the noise.

Stability analysis: 47% of tasks always pass, 19% always fail, 34% are flaky. The 39 always-fail tasks are a hard ceiling no prompt can breach. Arena scores were 5-13% lower than actual performance due to unit normalization bugs.

What would unlock higher performance: The 70% ceiling is a retrieval and extraction problem, not a prompt problem. We believe **sub-agents and reasoning LMs (RLMs)** are the key. A coordinator dispatching specialized sub-agents for retrieval, parsing, and computation would solve MiniMax's over-exploration. Our orchestrator pipeline was the right architecture but we ran out of time. The oracle ensemble proof (3 models = 75% vs 50% single) confirms that model diversity through sub-agents would yield substantial gains. The path forward: deterministic ingestion, structured extraction via sub-agents, deterministic computation – so the outer model only coordinates.

5. Feedback for Arena

What was confusing:

- `arena test` vs `arena submit` behave very differently – test doesn't copy skills/files, making validation impossible without actual submissions.
- The Goose Harbor recipe silently drops `max_turns`, `temperature`, `skills`, and `tarball` files. We spent days debugging features that were simply ignored.
- MCP never worked in arena despite working locally, with no error messages. `tool_definitions: null` was the only clue.

Things we wish worked:

- **Skills/summon** – not included in the Harbor recipe, making Goose skills a dead end.
- **Tarball file mounting** – files aren't mounted in the container. Our only workaround was base64-encoding scripts into `mcp_servers.args`.
- **Multiple harnesses per submission** – being locked to one harness+model per run made A/B testing slow.

What should be improved:

- **Trace retention** – traces purge after ~24h; should persist for the full competition.

- **Scoring transparency** – publish exact grading logic so teams can validate locally.
 - **Harness documentation** – a clear spec of what the harness uses vs. silently ignores would have saved 2-3 days.
 - **Submission variance** – 8-17 point variance makes evaluation hard. A “best of 3” or variance-adjusted leaderboard would help.
-

43 arena submissions. 9 architectural generations. ~4,400 task evaluations. 8 models tested. Best score: 184.5 at \$1.71. The winning system was a 28KB tarball with a 20-line prompt and no infrastructure. Hire Jwalin and Sanjay.